

Generating Resource-Efficient VHDL for Energy Reconstruction Using a High-Level Language

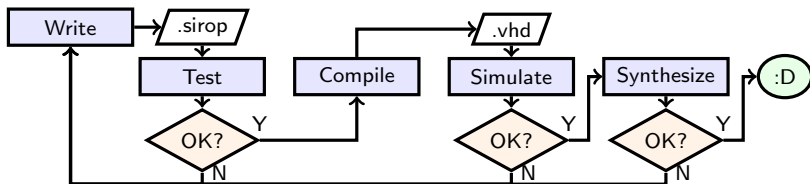
Louis Hildebrand Maheyer Shroff
Christophe Dubach Brigitte Vachon



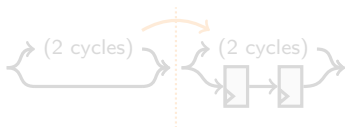
September 22, 2026

Recap: Project Goal

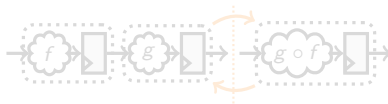
Goal: use a high-level language to develop resource-efficient VHDL implementations of energy reconstruction algorithms



Latency matching



Fusion

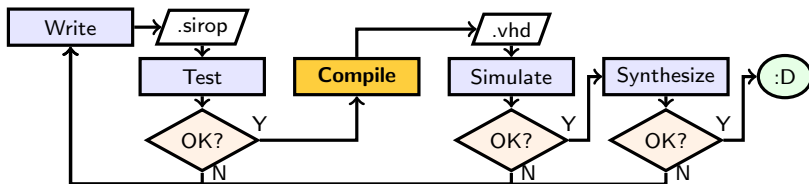


Fission

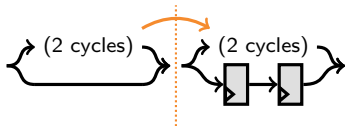
etc.

Recap: Project Goal

Goal: use a high-level language to develop resource-efficient VHDL implementations of energy reconstruction algorithms



Latency matching



Fusion

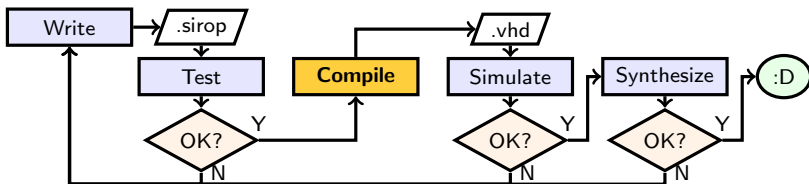
etc.



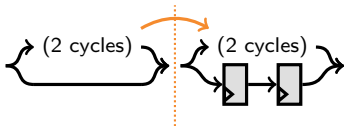
Fission

Recap: Project Goal

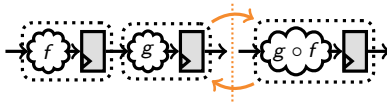
Goal: use a high-level language to develop resource-efficient VHDL implementations of energy reconstruction algorithms



Latency matching



Fusion



Fission

etc.

Previous Results, dacore FIR Filter

6-tap filter with hard-coded weights, targeting Agilex 7

Design	$f_{\max}$	ALMs	Δ ALMs	DSPs
Handwritten VHDL*	880 MHz	123 [‡]		3
Generated w/ SIROP [†]	823 MHz	118 [‡]	(-4%)	3

*LASP-dacore 9258fad (dacore-v8.2)

[†]SIROP v1.3.0

[‡]The numbers in the previous presentation (June 2026) are a bit different because I made some configuration mistakes (e.g., different part numbers for the target device). In this presentation, all numbers are for the G_BSP_AGI_NO_HW target.

Why the Difference in ALM Usage?

Different DSP pipelining settings!

- See presentation explaining DSP register packing:
<https://indico.cern.ch/event/1700332/>
- Updated LASP-dacore in merge request !143

Updated Results, dacore FIR Filter

6-tap filter with hard-coded weights, targeting Agilex 7

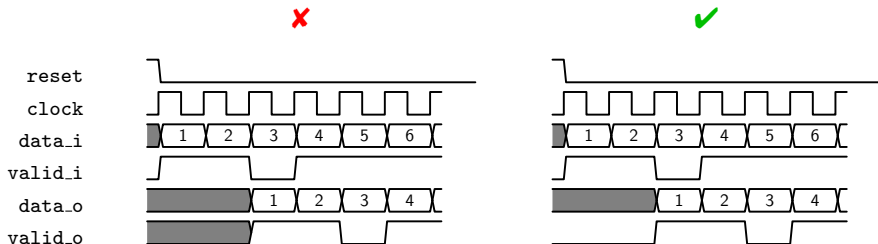
Design	$f_{\max}$	ALMs	Δ ALMs	DSPs
Handwritten, before !143*	880 MHz	123		3
Generated w/ SIROP [†]	823 MHz	118	(-4%)	3
Handwritten, after !143 [‡]	823 MHz	117	(-5%)	3

*LASP-dacore 9258fad (dacore-v8.2)

[†]SIROP v2.0.0

[‡]LASP-dacore 5ef49f2

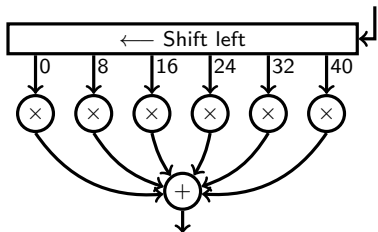
Initial valid Outputs



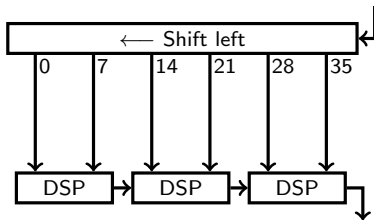
- It's good to ensure the `valid` field in `t_avst_stream` outputs is 0, even immediately after reset
 - See merge request [!142](#) in LASP-dacore
- SIROP can check such conditions (e.g., `valid` bit must be 0 for the first n cycles, where n is the entity's latency)

DSP Input Cascade

Intuitive FIR filter:



Using DSPs:



- This optimization also applies for neural networks (e.g., at CPPM)
- SIROP can automatically transform the simpler version into the resource-efficient version (by merging and shrinking redundant shift registers)!

Conclusion

- SIROP generates a resource-efficient FIR filter (using the Agilex DSPs) from concise source code
- Investigating differences between generated and handwritten VHDL led to improvements in robustness (!142) and resource-efficiency (!143)
- Next steps: more advanced energy reconstruction algorithms*
 - Neural networks (CPPM, Dresden)
 - Optimal filter with forward corrections (McGill)
- Future direction: faster exploration of neural net architectures (e.g., representation of weights, hypernetworks)



(SIROP
repository)

*<https://indico.cern.ch/event/1724142/#2-new-signal-reconstruction-me>

FIR Filter in SIROP (Part 1)

```
// Number of channels for time multiplexing (usually 8 or 12)
const M: u32 = 8
// How many inputs to check, when testing
const T: u32 = 136
const DEPTH: u32 = 6
const COEFFS: Vec[i18, DEPTH] =
    [-8_483:i18, 36_820:i18, -68_495:i18,
     44_585:i18, 17_867:i18, -9_379:i18]v
// Shift register size
const BUFSIZE: u32 = 1 + M * (DEPTH - 1)
// Number of pipeline registers to enable in the DSPs
const PIPELINE: u32 = 2

// See following slide for implementation...
```

FIR Filter in SIROP (Part 2)

```
// See previous slide for const declarations...
accelerator[no_handshake, library=dacore,
            head(adc_i)=(undefined:i18, false)]
dacore_fir_slv = (adc_i: Stm[(i18, bool), T]) =>
  let data =
    adc_i
    .StmMap( @(data, valid) => data )
    .StmSlide(BUFSIZE)
    // Select DEPTH elements, in steps of M
    .StmMap( buf => buf[:DEPTH:M] )
    .StmMapDot(StmCst(T-(BUFSIZE-1), COEFFS), PIPELINE)
    .StmMap( x => truncate18(x >> 15) )
  in
  let valid =
    adc_i
    .StmMap( @(data, valid) => valid, /*head=*/false)
    .StmSlide(BUFSIZE, /*head=*/false)
    .StmMap( buf => buf[:DEPTH:M].VecAll(), /*head=*/false)
  in
  StmZip(data, valid, /*head=*/(undefined:i18, false))
```
